Software Requirements Specification (SRS)

Project Name: CS Library Nuc Project

Team Members:
- Amilcar Armmand - Project Lead, Hardware Integration, Database, - armmanda1@southernct.edu
- Jose Gaspar Marin - Web Frontend, Backend (Admin and Remote Access) - gasparmarij1@southernct.edu
- Kenny Molina - Web Frontend (Core Kiosk) - molinak4@southernct.edu

Document Version: Draft v3.0
Last Updated: 3/30/26

---

1. Executive Summary

1.1 Project Overview

The CS Library Kiosk System is an automated, self-service checkout station designed for the Computer Science Department student lounge library. It replaces the current manual, paper-based system with a touchscreen kiosk that allows students to borrow and return books using barcode scanning and student ID card authentication.

1.2 Problem Statement

Computer Science students currently use a manual library system that uses paper checkout slips, provides no real-time inventory tracking, and offers no automated due date management. This leads to lost books, time wasted searching for unavailable resources, and having to manually track checkouts and returns.

1.3 Solution Overview

Our solution is a kiosk-first web application designed for a Raspberry Pi touchscreen deployment with USB barcode-scanner support and a companion web portal. The system provides instant book checkout/return, real-time inventory status, automatic ISBN lookup for cataloging, and a searchable digital catalog accessible through an intuitive interface.

1.4 Client Context

Client Organization: Computer Science Department, Southern Connecticut State University
Client Contact: IT Coordinator, Computer Science Department Faculty Advisor
Client Mission/Business: Providing resources and study space for computer science students
Project Sponsor: Faculty Advisor

The Computer Science Department needs an automated library system that reduces administrative burden, improves resource tracking, and provides better service to students. Success means having a reliable, easy-to-use system that operates independently in the student lounge with minimal maintenance requirements.

2. User Research & Personas

2.1 Research Methods

Research conducted:

- Observation or shadowing of current CS Lounge library usage
- Discussions with students using the CS Lounge
- Review the existing manual checkout workflow

Key Findings:

1. The CS Library shelf has a limited selection of books. There is room to add more books. We can add more books through donations which we plan to implement a way for students to give away CS books that they don't need anymore.
2. The only way to check in and out of books is to fill out a paper with the book name, the check in date and check out date.
3. There is more than enough room in the space to set up the device. There is also space to connect the NUC via Ethernet.

2.2 User Personas

Primary User Persona

Name & Role: Taylor, Computer Science Junior
Demographics: 21, Tech-savvy, comfortable with self-service systems, heavy library user
Organization Context: Full-time student, uses library resources for projects and exam preparation

Needs:

- Quick access to programming reference books
- Ability to check availability without searching shelves
- Simple checkout/return process

Pain Points:

- Can't tell if book is available without physically searching
- Manual checkout slips easily lost

Goals:

- Efficiently borrow and return books independently
- Access up-to-date catalog

Secondary User Persona
Name & Role: Professor James, CS Faculty Adminstrator
Demographics: 43, Faculty Member, strong technical proficiency

Needs:
- The ability to monitor inventory and checkout activity
- A easy way to add and remove books from the system

Pain Points:
- Manual tracking of books means there is a chance of missing items
- Can't see who has borrowed books
- The overhead is time-consuming

Goals:
- Reduce overhead an manual work of managing said library
- Ensure books are accounted for
- Improve the accessibility of books for students

Third User Persona

Name & Role: Kevin Rivera, Student Book Donor

Demographics: 22, CS Student or Student Alumni

Organization Context: Wants to donate technical books to help current students, interacting with kiosk to register books

Needs:
- Simple and guided way to donate books to CS Lounge using kiosk
- Scan ISBN and have details filled in
- Confirmation that books were donated/checked in to library database

Pain Points:
- No tracking of book donations

Goals:
- Donate books quick and efficiently
- Confirmation of books being donated in the system

---

3. User Stories

Core User Stories (Must Have)

Authentication & User Management

US001: As a student, I want to authenticate by scanning my student ID card so that I can use the system without remembering credentials.

Owner: Kenny

Acceptance Criteria:
- [ ] Scanner reader successfully detects student ID card
- [ ] System validates student is authorized to use library against the database withing 1 second
- [x] User profile is created automatically if not present
- [x] User is redirected to main dashboard after login
- [x] UI will display a notification if clear success or error

---

US002: As an administrator, I want to log in with username and password so that I can manage the system settings and data.

Owner: Jose

Acceptance Criteria:
- [ ] Secure admin login screen accessible from main menu
- [ ] Failed login attempts limited to 5 before lockout
- [ ] Role-based access to different admin functions
- [ ] Using bcrypt to validate the hashed password
- [ ] Role-based access to validate user type, admin/stuent before routing to dashboard

---

Book Management

US003: As a student, I want to check out a book by scanning its ISBN barcode so that I can borrow it quickly without paperwork.

Owner: Kenny

Acceptance Criteria:
- [ ] Barcode scanner reads ISBN successfully on first try 95% of time
- [ ] System displays book details (title, author, cover image) withing 1.5 seconds
- [x] Due date automatically set to 14 days from checkout
- [x] Transaction recorded with timestamp and user ID
- [x] Clear confirmation message with return date
- [x] Database updates the book_status to checked out instanly upon confirmation

---

US004: As a student, I want to return a book by scanning its barcode so that I can complete returns in seconds.

Owner: Kenny

Acceptance Criteria:
- [x] System recognizes book as currently checked out
- [ ] Scanner reads ISBN and queries the database
- [x] Database book status updates to available instantly upon scan
- [x] Book status updated to "available"
- [x] UI displays confirmation message

US005: As a student, I want to search the library catalog so that I can find books without browsing shelves.

Owner: Kenny

Acceptance Criteria:
- [x] Search by title, author, or keyword
- [x] UI displays results, displaying 12 books per page to prevent any overload
- [x] Status badge outputs available(green) or checked out(red) based on database state

User Account Features

US006: As a student, I want to view my borrowed books and due dates so that I can manage my returns.

Owner: Kenny

Acceptance Criteria:
- [x] Display list of currently checked out books
- [x] UI highlights books where due date less than current date in red
- [x] Option to renew books (if no holds)
- [x] Show borrowing history (last 6 months)
- [x] Total books checked out counter
- [x] System restricts the renew button if the current date to already past the due date

Administration Features

US007: As an administrator, I want to add new books to the system so that the catalog stays current.

Owner: Jose

Acceptance Criteria:
- [ ] Manual entry form for books without ISBN
- [ ] Open Library API request fetches and pares metadata with scan
- [ ] ISBN scan auto-populates metadata from API
- [ ] Option to categorize books (programming, theory, etc.)
- [ ] Upload cover images manually if needed
- [ ] Confirmation message after successful addition

US008: As an administrator, I want to view all checked-out books so that I can track library usage.

Owner: Jose

Acceptance Criteria:
- [ ] Dashboard showing all active checkouts
- [ ] Filter by due date (today, overdue, upcoming)
- [ ] Sort by borrower name or book title
- [ ] Export list to CSV for record keeping
- [ ] Overdue items highlighted with days overdue

US009: As an administrator, I want to manage user accounts so that I can manage library access.

Owner: Jose

Acceptance Criteria:

- [ ] Set borrowing limits per user
- [ ] View borrowing history per student
- [ ] Reset system for new semester

US010: As an administrator, I want to generate usage reports so that I can make data-driven decisions.

Owner: Jose

Acceptance Criteria:
- [ ] Monthly checkout statistics
- [ ] Most popular books report
- [ ] Usage patterns by time of day/day of week
- [ ] Export reports to CSV format
- [ ] Print summary reports

US011: As a Book Donor, I want to be able to donate a book using the kiosk by scanning it's ISBN, which should add the book to the CS Lounge Library Database.

Owner: Jose

Acceptance Criteria:
- [ ] Donor scans book ISBN at kiosk
- [ ] System fills in book metadata from ISBN
- [ ] Donor can also manually enter details if ISBN is invalid or missing
- [ ] Confirmation from system that book has been added to the database successfully
- [ ] Donation is logged with timestamp

US012: As a student, I want to reserve books that are currently checked out so that I can get them when returned.

Acceptance Criteria:
- [ ] Track books that have been checked out
- [ ] View currently checked out books marked in the system
- [ ] Schedule a pickup date, if pre-existing days are scheduled from other users
- [ ] Notify the student when the book has been returned, based on scheduled time, for pickup

---

Should Have (Post-MVP Enhancements)

US013: As a student, I want to receive email reminders about due dates so that I don't forget to return books.

Owner: Amilcar

Acceptance Criteria:

- [ ] Set the system to have a pre-due reminder to notify students that their book is almost due
- [ ] System emails them based on the pre-due reminder period set
- [ ] If the student does not return the book in time, set the system to create an overdue email to the user. Establish a set fee if overdue.

Could Have (Future Considerations)

US014: As a department chair, I want to access library statistics from a web dashboard so that I can monitor usage remotely.

Owner: Jose

Acceptance Criteria:
- [ ] System must track every action created by the end user (book check ins, book check outs, etc.)
- [ ] Dashboard is created to view statistics
- [ ] Department chair is given admin credentials and access

US015: As a student, I want to suggest books for purchase so that the collection meets student needs.

Owner: Jose

Acceptance Criteria:
- [ ] User fills out a form on the page detailing what books can be purchased
- [ ] System collects every form filled out
- [ ] Administrators view the form and review it for further consideration

---

4. Features & Requirements

4.1 Core Features

- Touchscreen Kiosk Interface - Touch-optimized UI with larger touch targets designed to allow a complete checkout in 4 or fewer taps.
- Student ID Authentication - Hardware-level barcode scaning of student ID cards that processes and queries the database.
- Barcode Book Scanning - ISBN scanning integration that reads an retrieves database metadata with each scan
- Real-time Inventory Management - Instant status updates showing book availability, checkout history, and due dates.
- ISBN API Integration - Automatic population of book details (title, author, cover) from online databases.

- Admin Management Dashboard - Comprehensive backend for managing books, users, and generating reports.
- Offline Operation Mode - Core functionality continues during network outages with sync when reconnected.
- Automated Due Date Tracking - Automatic calculation of return dates with overdue detection and reporting.

4.2 Technical Requirements

Authentication & Authorization

- Barcode scanner authentication for students via student ID cards
- Username/password authentication for administrators
- Session management with automatic timeout
- Role-based access control (student vs. admin vs. super-admin)

Data Management (CRUD Operations)

- Create: Add new books, register new users, record transactions
- Read: Search catalog, view borrowing history, generate reports
- Update: Checkout/return status, user information, book details
- Delete: Archive old records, remove lost books (soft delete)

Database & Storage

- Primary Database: SQLite3 for the current prototype (`data/cs_library.db`)
- Key Data Entities:
    - Users: student*id, name, email, password*hash, active, auto_provisioned
    - Books: isbn, title, author, cover, status, shelf
    - Loans: user*id, isbn, checked*out, due*date, returned, returned*date
    - Holds: user*id, isbn, status, pickup*date, created_at

- Future Migration: Move to a managed relational database only after the kiosk and portal workflows are stable on the current prototype.

4.3 Non-Functional Requirements

Performance

- Checkout/return transaction completes within 3 seconds
- Search results display within 2 seconds
- Support for 5+ concurrent users during peak hours
- System boots to ready state within 60 seconds from power on

Security

- Student ID numbers stored encrypted at rest
- No sensitive personal information stored in system
- Input validation on all user inputs
- Protection against SQL injection and XSS attacks
- Physical security measures for Raspberry Pi unit

Usability

- Touchscreen-optimized interface with 44px minimum touch targets
- High contrast display readable in various lighting conditions
- Clear audio feedback for successful scans
- Intuitive icon-based navigation
- Minimal training required for new users
- Single barcode scanner for both student IDs and books to simplify user interaction

Reliability

- 99% uptime during library operating hours (8 AM - 10 PM)
- Daily automated database backups

- Graceful recovery from power interruptions
- Error logging with timestamp and user context

Accessibility

- Screen reader compatibility for visually impaired users
- Keyboard navigation support for motor-impaired users
- High contrast mode option
- Adjustable text size (within limits of fixed display)

Hardware Requirements

- Single USB barcode scanner compatible with both ISBN barcodes and student ID barcodes
- Raspberry Pi 4 (4GB RAM minimum)
- 7-inch touchscreen display
- Optional receipt printer for transaction records
- Uninterruptible power supply (UPS) for graceful shutdown

---

5. System Design

5.1 Technology Stack

Frontend:

- Web Framework UI: NiceGUI (Python-based UI framework)
- Styling: Tailwind CSS
- Touchscreen Support: Web Browser Touch Events
- Architecture: Web Portal and Kiosk Site

Backend:

- Runtime: Python 3.11+ (optimized for Raspberry Pi 5)

- Framework: FastAPI (NiceGUI)
- Authentication: Student ID sign-in for kiosk, email/password for the web portal, bcrypt password hashing
- Validation: Pydantic for data validation
- Session State: NiceGUI user storage for kiosk and portal session persistence

Database:

- Primary Database: SQLite3 using the built-in `sqlite3` module
- Current Schema: users, books, loans, holds
- Local Asset Cache: book cover images cached under `assets/covers`
- Future Migration: Managed MySQL or PostgreSQL deployment after prototype stabilization

Hardware Integration:

- Current Prototype: keyboard-style USB scanner input and manual text-entry fallback
- Target Hardware: Raspberry Pi, USB barcode scanner, touchscreen display
- Scanner Strategy: scanner behaves like typed input in the current build, with dedicated hardware validation planned for the final phase
- Power Management: graceful shutdown and backup strategy planned for deployment phase

Deployment:

- Platform: Raspberry Pi 5 (on-premise)
- OS: Raspberry Pi OS (64-bit) Lite
- Process Management: Systemd for daemon management
- Service Orchestration: Supervisor or systemd services
- Logging: journald (systemd journal) with Python logging integration
- Backup: rsync + cron for automated backups

Development Tools:

- Version Control: Git + GitHub
- Package Management: pip + uv (faster Python package installer)

- Environment Management: pipenv or poetry
- Testing: pytest + pytest-asyncio
- IDE: VS Code with SSH remote development
- Debugging: pdb++, remote debugging with VS Code
- Documentation: Sphinx for API documentation

Third-Party Services:

- ISBN API: Open Library API via `httpx`
- No external authentication (self-contained system)

5.2 User Interface Design

Design Principles:

- Web-First Design: Responsive web application accessible via the Kiosk touchscreen or any remote browser.
- Keyboard/Touch Navigation: Support for both keyboard shortcuts and touch taps
- Modern UI: custom color schemes for readability
- Minimal Information: Display only essential information at each step
- Progressive Disclosure: Advanced options hidden until needed

Key Screens:

1. Kiosk Sign-In Screen

   Purpose: Authenticate a student for the kiosk workflow

   Key Elements:

   - SCSU-branded welcome layout
   - Student ID input with scanner-friendly focus behavior
   - Sign-in confirmation and validation messaging
   - Timestamp / system status display

   User Actions:

   - Scan or type a student ID

    - Submit sign-in
    - Continue into the kiosk dashboard

2. Web Portal Sign-In / Registration

   Purpose: Support remote access for email/password users

   Key Elements:

    - Email and password sign-in form
    - Registration page for new accounts
    - Validation feedback for duplicate or invalid credentials

   User Actions:

    - Register a new account
    - Sign in with email and password
    - Navigate to the browse-only dashboard

3. Catalog Dashboard

   Purpose: Central navigation hub for browsing and selecting actions

   Key Elements:

    - Top navigation tabs (Browse, Checkout, Return, My Books, Search)
    - Book card grid with cover images and availability badges
    - Search toggle and pagination controls
    - User identity and logout controls

   User Actions:

    - Browse the collection
    - Search by title or author
    - Switch between major workflow tabs

4. Checkout Workspace

   Purpose: Complete one or more checkout transactions

   Key Elements:

    - ISBN input for scanner or manual entry
    - Live cart summary
    - Book detail preview and due-date display
    - Checkout confirmation button

User Actions:

- Scan or type an ISBN
- Review the cart
- Confirm checkout

5. Return Workspace

Purpose: Process returned books quickly

Key Elements:

- Return scan/input field
- Returned-book cover and title confirmation
- Status message for successful or invalid returns

User Actions:

- Scan or type a book barcode / ISBN
- Confirm the returned item visually

6. My Books Screen

Purpose: Show active loans and borrowing history

Key Elements:

- Active loan list with due dates
- Renewal action for eligible books
- Borrowing history section
- Hold-related messaging for renewal restrictions

User Actions:

- Review active and returned books
- Renew eligible books
- Monitor due dates

7. Admin Dashboard (Planned)

Purpose: Manage books, users, reports, holds, and donations in a future release

Key Elements:

- Book and user management tools
- Checkout / usage reporting

- Hold queue visibility
- Donation intake workflow support

User Actions:

- Manage catalog data
- Review usage and loan activity
- Maintain hold and donation workflows

5.3 Database Schema

```
┌──────────┐          ┌──────────┐
│  User    │◄─────────┤  Loan    ├──────►│  Book    │
└──────────┘          └──────────┘          └──────────┘

     ▲                    ▲                    ▲
     │                    │                    │
     └──────────┐         │         ┌──────────┘
                ▼         ▼         ▼
                ┌──────────┐
                │   Hold   │
                └──────────┘
```

Entity: User

```
{
  "id": "Integer (primary key, autoincrement)",
  "student_id": "String (unique, required)",
  "name": "String (required)",
  "email": "String (unique, required)",
  "password_hash": "String (required)",
  "active": "Boolean-like integer (default: 1)",
  "auto_provisioned": "Boolean-like integer (default: 0)",
  "created_at": "Datetime (default current timestamp)"
}
```

Entity: Book

```
{
  "isbn": "String (primary key)",
  "title": "String (required)",
```

```
  "author": "String (required)",
  "cover": "String (default empty string)",
  "status": "String (default: 'Available')",
  "shelf": "String (default empty string)"
}
```

Entity: Loan

```
{
  "id": "Integer (primary key, autoincrement)",
  "user_id": "Integer (ref: users.id, required)",
  "isbn": "String (ref: books.isbn, required)",
  "checked_out": "Datetime (default current timestamp)",
  "due_date": "Datetime (required)",
  "returned": "Boolean-like integer (default: 0)",
  "returned_date": "Datetime (optional)"
}
```

Entity: Hold

```
{
  "id": "Integer (primary key, autoincrement)",
  "user_id": "Integer (ref: users.id, required)",
  "isbn": "String (ref: books.isbn, required)",
  "status": "String (default: 'pending')",
  "pickup_date": "Datetime (optional)",
  "created_at": "Datetime (default current timestamp)"
}
```

Planned future schema extensions for donation intake, admin reporting, and statistics remain in scope, but they are not part of the committed prototype schema yet.

5.4 System Architecture

Component Diagram:

```
┌──────────────────────┐      ┌──────────────────────────────┐
│ Kiosk UI / Web Portal│──────►│ NiceGUI + FastAPI App        │
└──────────────────────┘      │ - auth/session handling      │
```

```
                                 | - catalog workflows        |
                                 | - checkout / return / renew|
┌──────────────────────┐         | - Open Library lookups     |
│ USB Scanner / Manual │──────►  └────────────────────────────┘
│ Text Entry           │                      │
└──────────────────────┘                      ▼
                                 ┌──────────────────────────┐
                                 │ SQLite Data Layer        │
                                 │ users / books / loans /  │
                                 │ holds + local cover cache │
                                 └──────────────────────────┘
                                              │
                                              ▼
                                 ┌──────────────────────────┐
                                 │ Open Library API         │
                                 │ (metadata and cover data) │
                                 └──────────────────────────┘
```

Request/Response Flow:

```
Input Event: Student scans or types a student ID / ISBN, or uses the
touchscreen UI

Event Dispatch: NiceGUI page handler receives the event

Business Logic Processing:

    Input helpers normalize student IDs and ISBNs

    Database helpers query SQLite for users, books, loans, and holds

    Open Library metadata is requested when a scanned ISBN is not
already cached locally

    Checkout, return, renewal, and account provisioning logic updates
the data model

UI Update: NiceGUI components refresh the current view

Data Persistence: SQLite commits the transaction locally
```

```
Asset Cache: Cover images are cached locally when available

Response: Visual feedback shown on kiosk or web UI
```

Security Layer:

```
Authentication:

    Student: Student ID validation against the local database, with
auto-provisioning for kiosk use

    Admin: Password authentication with bcrypt hashing

    Session timeout and stricter lockout rules remain planned admin
hardening work

Authorization:

    Role-based permissions (student vs. admin)

    Browse-only restrictions for the remote portal

    Expanded admin role controls planned for final phase

Input Validation:

    Student ID and ISBN format validation

    SQL injection prevention via parameterized queries

    Input sanitization for all user inputs

    Boundary checking for all numeric inputs

Data Protection:

    Local SQLite data file permissions

    Secure logging (no sensitive data in logs)
```

```
    Planned automated backups before deployment

    File system permissions (restricted access to data files)
```

---

6. Implementation Plan

6.1 Sprint Breakdown (4 Sprints)

Sprint 1: Foundation & Authentication (Target: Progress Report 1 - March 2)

Timeline: Week 3-6
Goal: Project foundation and baseline authentication

Status: Complete

Tasks:
- Amilcar: Project planning, research, and early hardware investigation
- Kenny: Initial NiceGUI kiosk interface and student sign-in flow
- Jose: Baseline data model, user registration, and authentication support

Deliverables:
- Working login and dashboard foundation

Sprint 2: Core Feature Development

Timeline: Week 7-9
Goal: Library Workflows and external integrations

Status: Complete

Tasks:
- Amilcar: Metadata integration planning and workflow validation
- Kenny: Search, manual fallback input, checkout, return, and My Books workflows
- Jose: Registration/login improvements, seed data, and backend circulation support

Deliverables:

- Functional checkout / return flow with Open Library metadata support

Sprint 3: Feature Completion & Enhancement (Target: Progress Report 2)

Timeline: Week 10-12
Goal: Polish the prototype for Progress Report II

Status: Mostly complete

Tasks:
- Amilcar: Continue hardware planning and deployment research
- Kenny: Refine kiosk dashboard flow, renewals, and responsive UI behavior
- Jose: Improve portal auth flow, data persistence, and supporting backend logic

Deliverables:
- PR2-ready prototype with kiosk and web portal workflows

Sprint 4: Testing, Polish & Deployment

Timeline: Week 13-14
Goal: Final-phase completion and validation

Status: In progress

Tasks:
- Team: Admin dashboard completion, hold / donation workflow work, hardware validation, regression testing, and final documentation

---

7. Risk Assessment

Technical Risks

Risk 1: SD Card Corruption
- **Impact:** High
- **Likelihood:** Medium
- **Mitigation:** Raspberry Pi SD cards are prone to failure under heavy loads. We will implement daily automated backups that mirror the database to a USB hard drive.

Risk 2: Network Dependencies and Database Outages
- **Impact:** High
- **Likelihood:** Low
- **Mitigation:** The prototype uses a local SQLite database so students can still browse the catalog even if external network access is unavailable.

Risk 3: Campus IT Approval
- **Impact:** High
- **Likelihood:** Low
- **Mitigation:** Waiting for SSO credentials from IT, team is using a mock authentication system until IT provides credentials then we swap for integration.

Risk 4: Barcode Error Hardware Failure
- **Impact:** Medium
- **Likelihood:** High
- **Mitigation:** The Kiosk UI includes the ability for manual text-entry fall back. If the physical USB scanner drops, you could still manual touchscreen typing.

Risk 5: Hardware Integration / Delays
- **Impact:** High
- **Likelihood:** Low
- **Mitigation:** The web app includes manual text input so hardware does not block feature development. The USB scanner behaves like keyboard input, so the kiosk software can still be tested on laptops before final Pi deployment.

8. Success Metrics

- System uptime of 99% during testing phase specifically in school hours
- Book checkout transaction time under 20 seconds
- Have all available books in CS Library cataloged into the system

---

9. Testing Strategy

9.1 Testing Overview

- The project is tested at three levels: unit, integration, and system.
- The current PR2 workflow uses smoke tests against a temporary copy of the SQLite database so application data is not corrupted during verification.
- Final-phase validation will expand this into a repeatable regression suite before delivery.

9.2 Unit Testing

Targeted units include:

- student ID and ISBN normalization / validation helpers
- user registration and email authentication logic
- checkout, renewal, and return logic
- local catalog lookup behavior

Planned tooling:

- `pytest`
- `pytest-asyncio`

9.3 Integration Testing

Integration tests focus on:

- registration followed by successful authentication
- checkout followed by correct appearance in My Books
- return followed by correct borrowing-history updates
- kiosk auto-provisioning for unknown student IDs
- Open Library lookup when an ISBN is not already in the local catalog

9.4 System Testing

System tests cover:

- web portal sign-in page load
- registration page navigation
- kiosk browse / checkout / return workflows
- invalid-input handling
- scanner validation on target hardware

9.5 Current PR2 Status

- Unit smoke checks executed: 6 passed
- Integration smoke checks executed: 4 passed
- System smoke checks executed: 2 passed
- Remaining work: broader end-to-end UI coverage, hardware scanner validation, and final regression automation

10. Appendix

A. Glossary

- **ISBN:** International Standard Book Number - A unique numeric commercial book identifier
- **NiceGUI:** Python-based web-UI
- **SQLite:** Embedded relational database used by the current prototype

B. References

Technical References
- Raspberry Pi Foundation. (2024). Raspberry Pi Documentation. https://www.raspberrypi.com/documentation/
- Open Library API Documentation. https://openlibrary.org/developers/api
- FastAPI Documentation. https://fastapi.tiangolo.com/
- NiceGUI Documentation: https://nicegui.io/documentation

C. Change Log

Date	Version	Changes	Author
Feb 6, 2026	v1.0	Initial submission	Team
Mar 2, 2026	v2.0	PR1 revisions and requirement updates	Team
Mar 30, 2026	v3.0	PR2 revisions, aligned architecture and schema to current prototype, refreshed sprint plan, and added detailed testing strategy	Team

---

Document Status: Review
Next Review Date: April 6, 2026

Prepared by: CS Library Team
Course: CSC400 - Computer Science Project Seminar
Semester: Spring 2026